

Project Guide: Building a System-Design-Grade URL Shortener

This is a guide to the *thinking*, not the code. Every section tells you what to build and why, and what questions to answer before you write it — the actual implementation is yours to write, which is the point: you'll understand every line because you reasoned through it first.

0. Before you write anything: define done

Write a one-page scope doc (even just a `SCOPE.md` in your repo) answering:

- What are the exact endpoints? (e.g., create short link, redirect, get click stats for a link, delete a link)
- Who's the user — anonymous, or logged-in only? This decides whether you need auth from day one.
- What's v1 (must-ship) vs v2 (nice-to-have)? Rate limiting and analytics can be v1.5 if you're tight on time — a working core beats an ambitious unfinished one.

This step feels like overhead. It isn't — it's what stops the project from sprawling, and it's the first artifact an interviewer might ask to see.

1. Data modeling — think in questions, not tables

Before you write a single SQLAlchemy model, answer these on paper:

For the URL entity:

- What uniquely identifies a short link? (the short code itself — so it needs a unique constraint, and probably an index, since every single redirect does a lookup on it)
- Does a link expire? If yes, you need an `expires_at` field and a decision: do you check expiry on every read, or run a cleanup job? Either is defensible — the point is to *decide* and be able to explain why.
- Is a link owned by a user, or anonymous? If owned, that's a foreign key and it changes your auth story.
- What happens if two requests generate the same short code at the same time? This is a real race condition — decide whether you handle it with a DB unique constraint + retry, or a different code-generation strategy that avoids collisions altogether (more on this below).

For the click/analytics entity:

- What do you actually need to know about a click? Timestamp is obvious. Referrer and a hashed IP (never store raw IPs — that's a privacy problem, and hashing it is the kind of small decision that signals maturity) are useful for the analytics feature.
- Should this be one table that grows forever, or do you think about partitioning/archiving old data? You don't have to *build* partitioning, but knowing

you'd need it at scale is a system-design talking point.

Deliverable for this section: a schema diagram (even hand-drawn or an ASCII sketch in your README) with every field, its type, and which fields are indexed — before any migration file exists.

2. Short code generation — this is a real design decision, not a detail

You have three real options. Understand the tradeoff of each rather than picking the first one that works:

1. **Auto-increment ID + base62 encode it.** Simple, guaranteed no collisions, but leaks information (sequential IDs reveal how many links exist, and are guessable).
2. **Random string, check DB for collision, retry if taken.** No information leakage, but at scale, collision-checking is an extra DB read on every write, and collision probability needs actual math to reason about (birthday paradox) — at what code length does collision become negligible?
3. **Hash the long URL (e.g., first N chars of a hash) and truncate.** Deterministic — same input always gets same output, which can be a feature (idempotent shortening) or a bug (can't shorten the same URL twice for different users) depending on your requirements.

What to actually do: pick one, but write down in your README *why* you rejected the other two. That comparison is worth more than the code itself in an interview.

3. The redirect path — where performance actually matters

The redirect endpoint (`(short code → long URL → 302 redirect)`) is the hottest path in this whole system — it's the one endpoint that gets hit far more than link creation ever will. Treat it differently from the rest of your CRUD:

- **Why caching matters here specifically:** every redirect is a read of something that rarely changes. That's the textbook case for a cache-aside pattern. Before implementing, be able to answer: what's your cache key? What's the TTL, and why that number? What happens on a cache miss — do you cache the result of that DB read immediately, or only cache "hot" links?
 - **Invalidation is the hard part, not the caching itself.** If a link is deleted or edited, your cache now holds stale data. Decide: do you actively invalidate (delete from cache on write), or rely on TTL expiry and accept some staleness? Both are valid — but only if you can explain the tradeoff (staleness window vs. extra write complexity).
 - **Write this reasoning into your README as you build it,** not after — it's easy to forget your own reasoning once the code just "works."
-

4. Rate limiting — a DSA problem wearing a backend costume

Rate limiting the `/shorten` endpoint (to stop abuse) is worth building for real, not stubbing out, because the algorithm choice is a genuine data-structures decision:

- **Fixed window counter:** simplest — count requests per IP in the current time window, reset on window boundary. Easy to implement, but has a burst problem at window edges (explain this edge case in your README — it's a common follow-up question).
- **Sliding window / sliding log:** more accurate, more complex, more memory. What data structure would you use to track a sliding log efficiently? (Hint: think about what you'd trim and when — this connects directly to queue-like structures you'd see in DSA practice.)
- **Token bucket:** allows bursts up to a limit while maintaining a steady refill rate — this is what most production systems actually use (AWS, Stripe, etc.), and it's worth understanding even if you implement the simpler fixed window first.

What to build: start with fixed window (it's genuinely fine for v1), but document in your README which algorithm you'd switch to at scale and why. Implementing the harder one later, once the simple version works, is a legitimate and honest progression to describe in an interview.

5. Analytics — where you deliberately use raw SQL

Your NutriPlot AI project already demonstrates ORM usage (SQLAlchemy). This project's analytics feature is your chance to show you can also reason in raw SQL, which is a different and important skill:

- Design 2-3 real aggregation queries: clicks per day for a link, top referrers, most-clicked links for a user.
 - Write these as raw SQL, not through the ORM. Before writing each query, think through: what would the query plan look like? Which columns need an index to make this fast? (This is where your `clicks` table indexes from Section 1 get tested.)
 - If you want to push further: try writing one query using a window function (e.g., ranking links by click count) — window functions come up often in both interviews and real analytics work, and this project gives you a legitimate place to use one.
-

6. Auth — only if it earns its place

Don't bolt on auth just because your other project had it. Decide first: does this project actually need user accounts? If links are per-user (so users can manage their own links and see their own analytics), yes — reuse the JWT pattern you already know from NutriPlot AI rather than learning a new auth approach at the same time as everything else here. If not, skip it and note in your README that it's a deliberate v2 scope decision, not an oversight.

7. Testing — what's actually worth testing here

Don't aim for 100% coverage for its own sake. Prioritize:

- The short-code generation logic (collision handling, especially)
- The redirect path, including the cache-hit and cache-miss branches separately
- The rate limiter's boundary conditions (exactly at the limit, one over)

These are the places bugs actually hide, and testing them specifically (rather than trivial CRUD) shows you understand where risk lives in this system.

8. The README — treat this as the actual deliverable

Most of the value of this project, for interview purposes, lives in the README, not the code. Structure it like a lightweight design doc:

1. **Problem & scope** — what this does and doesn't do
 2. **Architecture diagram** — even a simple boxes-and-arrows sketch (client → API → cache → DB)
 3. **Key design decisions**, each with a one-paragraph "why," specifically:
 - Short code generation strategy chosen and rejected alternatives
 - Caching strategy and invalidation approach
 - Rate limiting algorithm chosen and what you'd use at scale
 4. **What breaks at 10x traffic** — a short, honest section on where the current design would need to change (e.g., "the single Postgres instance becomes the bottleneck; I'd introduce read replicas for the redirect path since it's read-heavy, and consider sharding by short-code prefix if writes also become a bottleneck"). You don't need to build this — reasoning about it convincingly is the point.
 5. **What you'd do with more time** — shows self-awareness, which interviewers read as maturity.
-

Suggested build order (do NOT build top to bottom of the sections above — build in this order instead)

1. Scope doc + schema sketch (Sections 0 & 1)
2. Basic CRUD API: create + redirect, no caching, no rate limiting (get this fully working and tested first)
3. Add caching to the redirect path (Section 3)
4. Add rate limiting to the create path (Section 4)
5. Add analytics with raw SQL (Section 5)
6. Add auth, if you decided it's in scope (Section 6)
7. Write the README properly (Section 8) — do this as you go, not all at the end, or you'll forget your own reasoning

8. Test, containerize, deploy

Each stage should be a working, committed state — resist building everything at once and debugging it all together at the end.

A note on pacing against your 3-month timeline

This project, done properly, is roughly 2-3 weeks of evenings/weekends if you're also doing DSA daily. If you notice yourself still polishing the caching layer in week 4, that's a signal to timebox it and move to writing the README and deploying — a slightly-less-polished finished project with a strong README beats an unfinished "perfect" one every time, especially against a hard deadline.